## Part A

**Ex: 3**

**Title:** Sorting elements in a list by breaking them into sub-lists.

**Problem Description:** Run the program for varied values of n to demonstrate the behaviour of the algorithm in the Worst, Best and Average Cases. Record the time taken to sort. Plot a graph of the time taken versus *n* on graph sheet. The elements can be read from a file or can be generated using the random number generator for large values of n.

**Method:** A Divide and Conquer- Recursive algorithm implementation- Merge Sort.

**Theory Reference:** Module 2

***Explanation***

Mergesort sorts a given array $A[0..n-1]$ by

- dividing it into two halves $A[0..\lfloor n/2 \rfloor - 1]$ and $A[\lfloor n/2 \rfloor ..n-1]$,
- sorting each of them recursively, and then
- merging the two smaller sorted arrays into a single sorted one.

The ***merging*** of two sorted arrays can be done as follows.

- Two pointers (array indices) are initialized to point to the first elements of the arrays being merged.
- The elements pointed to are compared, and the smaller of them is added to a new array being constructed;
  - the index of the smaller element is incremented to point to its immediate successor in the array it was copied from.
  - This operation is repeated until one of the two given arrays is exhausted, and then the remaining elements of the other array are copied to the end of the new array.

***Algorithm***

**ALGORITHM** *Mergesort(*$A[0..n-1]$*)*

//Sorts array $A[0..n-1]$ by recursive mergesort

//Input: An array $A[0..n-1]$ of orderable elements

//Output: Array $A[0..n-1]$ sorted in nondecreasing order

**if** $n > 1$

copy $A[0..\lfloor n/2 \rfloor - 1]$ to $B[0..\lfloor n/2 \rfloor - 1]$

copy $A[\lfloor n/2 \rfloor..n-1]$ to $C[0..\lceil n/2 \rceil - 1]$

Mergesort($B[0..\lfloor n/2 \rfloor - 1]$)

```
    Mergesort(C[0.. ⌈n/2⌉ − 1])
    Merge(B, C, A)
```

**ALGORITHM** Merge(B[0..p − 1], C[0..q − 1], A[0..p + q − 1])

//Merges two sorted arrays into one sorted array

//Input: Arrays B[0..p − 1] and C[0..q − 1] both sorted

//Output: Sorted array A[0..p + q − 1] of the elements of B and C

```
i ←0; j ←0; k←0
while i <p and j <q do
    if B[i]≤ C[j ]
        A[k]←B[i];
        i ←i + 1
    else
        A[k]←C[j ];
        j ←j + 1
    k←k + 1
if i = p
    copy C[j..q − 1] to A[k..p + q − 1]
else
    copy B[i..p − 1] to A[k..p + q − 1]
```

***Program***

```
import java.util.Scanner;
import java.util.Random;
public class MergeSort {
   public static int count=0;

   static void merge(int b[], int c[],int a[])
   {
      //Merges two sorted arrays into one sorted array
      //Input: Arrays b and c, both sorted
      //Output: Sorted array a of the elements of b and c

      int i=0,j=0,k=0;
```

```
    int p=b.length;
    int q=c.length;
    while((i<p) && (j<q))
    {
        count++;
        if(b[i]<=c[j])
            a[k++]=b[i++];
        else
            a[k++]=c[j++];
    }
    //Copy the remaining array elements from any one of sub-array
    while(i<p)
        a[k++]=b[i++];
    while(j<q)
        a[k++]=c[j++];
}

static void mergesort(int a[])
{
    //Sorts array a by recursive mergesort
    //Input: An array a of orderable elements
    //Output: Array a sorted in nondecreasing order

    int n = a.length;
    if(n>1)
    {
        int p=(int)Math.floor(n/2);
        int q=(int)Math.ceil(n/2);
        int b[]=new int[p];
        int c[]=new int[q];
        System.arraycopy(a,0,b,0,p);
        System.arraycopy(a,p,c,0,q);
        mergesort(b);
        mergesort(c);
        merge(b,c,a);
    }
}

public static void main(String[] args)
```

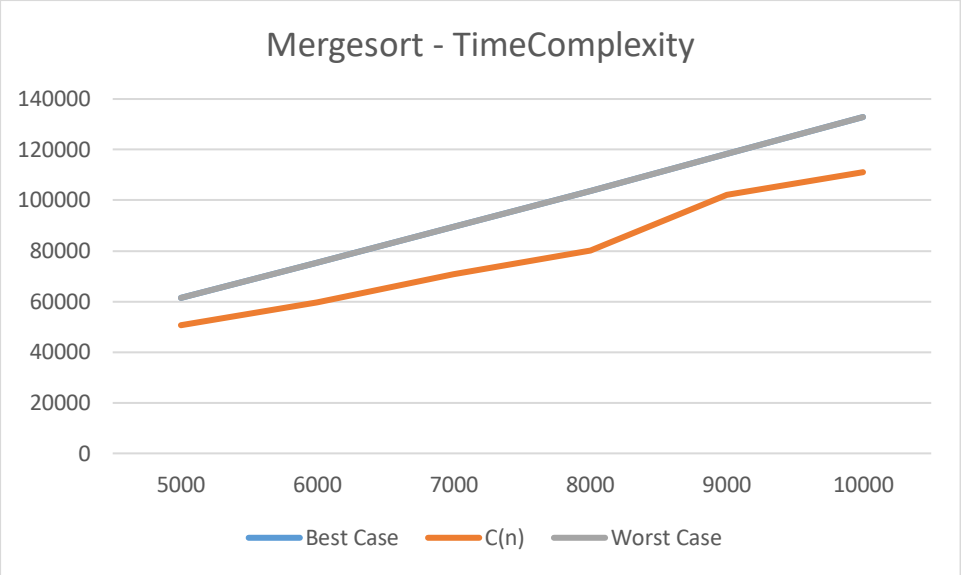
```
    {
        int n;
        Scanner s = new Scanner(System.in);
        System.out.println("Enter the no. of elements");
        n = s.nextInt();
        int a[] = new int[n];

        //use random class object to generate random values
        Random r = new Random();
        System.out.println("intput numbers");
        for(int i=0;i<n;i++)
        {
            a[i]=r.nextInt(10000);
            System.out.print(a[i] + " ");
        }

        mergesort(a);
        System.out.println("\n\nSorted numbers are");
        for(int i=0;i<n;i++)
            System.out.print(a[i] + " ");
        System.out.println("\nNo. of basic operations are " + count);
    }
}
```

**Output:**

| n | Best Case nlogn | Basic Operation Count | Worst Case nlogn |
|---|---|---|---|
| n | Best Case | C(n) | Worst Case |
| 5000 | 61439 | 50634 | 61439 |
| 6000 | 75304 | 59577 | 75304 |
| 7000 | 89412 | 70876 | 89412 |
| 8000 | 103726 | 80219 | 103726 |
| 9000 | 118221 | 102079 | 118221 |
| 10000 | 132877 | 111093 | 132877 |




public static void arraycopy(Object source_arr, int sourcePos,
Object dest_arr, int destPos, int len)

Parameters :

source_arr : array to be copied from

sourcePos : starting position in source array from where to copy

dest_arr : array to be copied in

destPos : starting position in destination array, where to copy in

len : total no. of components to be copied.